Elasticsearch Security Configuration Report

This report documents the security configurations applied to the Elasticsearch Docker container setup and the `elasticindexer` configuration files, detailing the technical reasoning behind these changes.

---

1. Executive Summary

To align the repository with production-grade security standards and comply with security audits, authentication has been enabled on the local Elasticsearch service. The `elasticindexer` config has been synchronized to utilize these credentials instead of running with the development mode security bypass.

---

2. The "Username and Password Required" Startup Error

Error Log

```
ERROR[2026-05-21 17:37:50.703]  elasticsearch username and password are required
(set allow-insecure-no-auth-dev = true to opt out for local development) while
loading the preferences config file
```

Why we are getting this error

This error is triggered by the indexer's config validator (`loadClusterConfig` in `main.go`). Under the default "fail-closed" security policy: 1. If the configuration has an empty `username` and `password` , the validator blocks execution. 2. To bypass this for insecure local development, you must explicitly opt in by setting `allow-insecure-no-auth-dev = true` in the configuration. 3. If credentials are empty and the opt-in flag is `false` (or missing), the validator throws this error and stops the service.

---

3. Detailed Modifications

A. Scripts Configuration (`scripts/script.sh`)

The Docker container launch configuration was modified to enable Elasticsearch's built-in security features and set an initial superuser password. Additionally, all index maintenance `curl` commands in the `delete()` function were updated to include basic credentials to prevent `401 Unauthorized` failures.

Code Diff

```
   docker rm -f ${IMAGE_NAME} 2> /dev/null
   docker run -d --name "${IMAGE_NAME}" -p 9200:9200  -p 9300:9300 \
-   -e "discovery.type=single-node" -e "xpack.security.enabled=false" -e
"ES_JAVA_OPTS=-Xms512m -Xmx512m" \
+   -e "discovery.type=single-node" -e "xpack.security.enabled=true" -e
"ELASTIC_PASSWORD=myPassword" -e  "ES_JAVA_OPTS=-Xms512m -Xmx512m" \
     docker.elastic.co/elasticsearch/elasticsearch:${ES_VERSION}

 delete() {
    for str in ${INDICES_LIST[@]}; do
-      curl -XDELETE http://localhost:9200/$str-000001
-      curl -XDELETE http://localhost:9200/$str
-      curl -s -o /dev/null -w "%{http_code}" -X GET localhost:9200/_ilm/policy/
$str-policy | grep -q 200 && curl -X DELETE localhost:9200/_ilm/policy/$str-policy
+      curl -u elastic:myPassword -XDELETE http://localhost:9200/$str-000001
+      curl -u elastic:myPassword -XDELETE http://localhost:9200/$str
+      curl -u elastic:myPassword -s -o /dev/null -w "%{http_code}" -X GET
localhost:9200/_ilm/policy/$str-policy | grep -q 200 && curl -u elastic:myPassword
-X DELETE localhost:9200/_ilm/policy/$str-policy
       echo
    done

-  curl -XDELETE http://localhost:9200/_template/*
+  curl -u elastic:myPassword -XDELETE http://localhost:9200/_template/*
   echo
 }
```

Rationale

- **`xpack.security.enabled=true`** : This tells the Elasticsearch cluster to require authentication. Unauthenticated requests to the cluster will now be rejected with HTTP `401 Unauthorized`.
- **`ELASTIC_PASSWORD=myPassword`** : This sets the password for the default administrator account (`elastic`).
- **`-u elastic:myPassword` flag**: Authenticates the curl maintenance operations so the script is still authorized to inspect and delete indices/templates/policies under active security.

B. Indexer Preferences (**`cmd/elasticindexer/config/prefs.toml`**)

The configuration for the database client was updated to provide valid credentials and enforce security check validation.

Code Diff

```
     [config.elastic-cluster]
         url = "http://localhost:9200"
-        username = ""
-        password = ""
-        allow-insecure-no-auth-dev = true
+        username = "elastic"
+        password = "myPassword"
+        allow-insecure-no-auth-dev = false
         bulk-request-max-size-in-bytes = 4194304 # 4MB
         num-writes-in-parallel = 1
```

Rationale

- **`username` and `password`** : Provides the indexer with the credentials set in `scripts/script.sh`.
- **`allow-insecure-no-auth-dev = false`** : Disables the local-only security bypass in the codebase. By setting this to `false`, we ensure the indexer must supply a username and password to start up successfully.

C. Integration Test Helper Configuration (`integrationtests/utils.go`)

Since integration tests connect to the local Elasticsearch container to run database assertions, they also failed with HTTP `401 Unauthorized` errors when security was enabled. The helper functions were modified to retrieve and provide these credentials.

Code Diff

```
 func createESClient(url string) (elasticproc.DatabaseClientHandler, error) {
+   username := os.Getenv("ELASTIC_USERNAME")
+   if username == "" {
+       username = "elastic"
+   }
+   password := os.Getenv("ELASTIC_PASSWORD")
+   if password == "" {
+       password = "myPassword"
+   }
+
    return client.NewElasticClient(elasticsearch.Config{
        Addresses: []string{url},
+       Username:  username,
+       Password:  password,
        Logger:    &logging.CustomLogger{},
    })
 }

 func getIndexMappings(index string) (string, error) {
    u, _ := url.Parse(esURL)
    u.Path = path.Join(u.Path, index, "_mappings")
-   res, err := http.Get(u.String())
-   if err != nil {
-       return "", err
-   }
+
+   req, err := http.NewRequest("GET", u.String(), nil)
+   if err != nil {
+       return "", err
+   }
+
+   username := os.Getenv("ELASTIC_USERNAME")
+   if username == "" {
+       username = "elastic"
+   }
+   password := os.Getenv("ELASTIC_PASSWORD")
+   if password == "" {
+       password = "myPassword"
+   }
+   req.SetBasicAuth(username, password)
```

```
+
+   httpClient := &http.Client{}
+   res, err := httpClient.Do(req)
+   if err != nil {
+       return "", err
+   }
+   defer res.Body.Close()
```

Rationale

- **Dynamic Credentials with Fallbacks**: `createESClient` and `getIndexMappings` retrieve credentials from `ELASTIC_USERNAME` and `ELASTIC_PASSWORD` environment variables, defaulting to `elastic` and `myPassword` to support seamless local developer test execution out-of-the-box.
- **Basic Authentication**: `SetBasicAuth` was added to standard HTTP requests to properly transmit authentication headers.

---

4. Underlying Technical Context

Why did these changes happen?

1. **Production-Ready Standards**: The fork's codebase implements a "fail-closed" security policy. The application requires credentials by default. By enabling credentials locally, we mirror a secure production configuration.
2. **Configuration Validation**: In Go, the loader checks the configuration struct at startup: `go ec := cfg.Config.ElasticCluster if ec.UserName == "" && ec.Password == "" && !ec.AllowInsecureNoAuthDev { return cfg, errors.New("elasticsearch username and password are required") }` Setting credentials and disabling `allow-insecure-no-auth-dev` verifies that the indexer successfully passes this validation check and authenticates against a secure cluster.

---

5. Verification and Reversion

Reverting to Password-less Mode

If you prefer a simpler developer setup without passwords, you can revert both configurations: 1. In `scripts/script.sh`, set `-e "xpack.security.enabled=false"` and remove the `ELASTIC_PASSWORD` environment variable. 2. In `prefs.toml`, clear the `username` and `password` fields, and set `allow-insecure-no-auth-dev = true`.

---

6. Local Development vs. Production Deployment Configuration

The table below highlights what configurations should be kept for local development versus what must be changed for production deployment:

Configuration Parameter	Local Development Recommendation	Production Deployment Recommendation	Rationale / Explanation
`xpack.security.enabled`	`false` (default) or `true`	**Must be `true`**	Disabling security locally reduces configuration overhead. In production, security must be enabled to prevent unauthorized access.
`allow-insecure-no-auth-dev`	`true` (if security is disabled)	**Must be `false`**	Acts as an explicit configuration gate to ensure the indexer doesn't accidentally run without credentials in production.
`username` & `password`	Can be empty (if security is disabled)	**Must be populated** with secure credentials	Local setups don't require access control, but production deployment must have authenticated client sessions.
Connection URL	`http://localhost:9200`	Secure HTTPS Endpoint (e.g. `https://es-cluster.internal:9200` or a managed service endpoint)	Production networks must use TLS/SSL to encrypt database traffic and prevent credential sniffing or data interception.

What to keep in Local Development:

- **Allow-insecure flag**: You can keep `allow-insecure-no-auth-dev = true` to allow local testing against default Docker instances without basic auth setup.
- **Plain HTTP**: Keep `http://localhost:9200` (non-TLS/plain HTTP connection) since traffic does not leave the local developer machine.
- **Security Disabled**: Keep `xpack.security.enabled=false` in local Docker scripts to avoid managing credential rotation during local testing.

What must change in Production Deployment:

- **Mandatory Authentication**: `allow-insecure-no-auth-dev = false` (or omit/remove it entirely, as it defaults to false).
- **Secure Credentials**: Provide strong, rotated credentials in `username` and `password` inside `prefs.toml`. Avoid using the root `elastic` account; configure a least-privilege role/user dedicated to indexing write operations.
- **Transport Security**: Connect using secure transport schemes (`https://`) to secure database traffic over the network.
- **Firewall/Network Policies**: Ensure the Elasticsearch cluster is behind a firewall/VPC and only accessible by authorized indexers and proxies.